

AI LAB 99

Machine Learning Track — Week 3, Module 5

Feature Engineering & Preprocessing Report

Preparing a Clustering-Ready Dataset for Customer Personality Analysis

Dataset: Customer Personality Analysis (customer_personality_cleaned.csv)

August 2026

Prepared for AI Lab 99 | Customer Segmentation Curriculum

Table of Contents

Table of Contents	2
Introduction & Dataset Overview	3
Task 1 — Customer Feature Creation.....	3
Task 2 — Categorical Feature Encoding	4
Task 3 — Feature Selection	5
Task 4 — Skewness and Feature Transformation.....	5
Task 5 — Feature Scaling	7
Task 6 — Feature Engineering Documentation (Feature Dictionary)	8
Task 7 — Preprocessing Pipeline Development.....	8
Task 8 — Final Dataset Validation.....	9
Limitations & Recommendations	10

Introduction & Dataset Overview

This report documents the feature engineering and preprocessing work performed on the Customer Personality Analysis dataset, continuing from the cleaned output of the Week 3 Module 4 EDA stage. The goal across the eight tasks below is to transform a cleaned but raw-featured customer table into a fully numeric, scaled, and validated dataset ready for clustering / customer segmentation.

Input file	customer_personality_cleaned.csv
Input shape	2,240 rows × 33 columns
Reference year	2014 (used for all age/tenure calculations)
Output file	Customer_Personality_Final_Dataset.csv
Final shape	2,038 rows × 40 columns (after de-duplication)

Task 1 — Customer Feature Creation

Thirteen behavioral and demographic features were engineered from the raw columns to give downstream clustering more direct, interpretable signal than the raw transactional fields alone.

Feature	Formula / Calculation	What It Captures
Customer_Age	2014 – Year_Birth	Converts birth year into a directly usable age in years.
Customer_Tenure	2014 – year(Dt_Customer)	Years since enrollment; a proxy for customer loyalty/relationship length.
Total_Children	Kidhome + Teenhome	Combines both child-count fields into one household-composition measure.
Family_Size	Partner (2 if Married/Together else 1) + Total_Children	Approximates total household size, a key driver of basket size and spending.
Total_Spending	Sum of MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds	Overall two-year spend across all product categories — the core value metric per customer.
Total_Purchase	Sum of purchase-count columns across channels	Total transaction count, a proxy for purchase frequency and engagement.
Total_Campaign	Sum of accepted marketing campaign flags	Counts how many past campaigns a customer responded to — a marketing-responsiveness score.
Average Spending per Purchase	Total_Spending / Total_Purchase	Average basket value; distinguishes frequent-small-basket shoppers from infrequent-big-basket shoppers.
Digital_Engagement	NumWebPurchases + NumWebVisitsMonth	Combines online purchases and site visits into one digital-channel engagement score.
Deal_Dependency	NumDealsPurchases / Total_Purchase	Share of purchases made on a discount/deal — flags price-sensitive customers.

Feature	Formula / Calculation	What It Captures
Preferred_Shopping_Channel	arg-max of {Web, Catalog, Store} purchase counts	Categorical label for the channel a customer relies on most, useful for channel-targeted marketing.
Product_Preference	arg-max of the six Mnt* spending columns	Categorical label for a customer's top spending category, useful for product-targeted offers.
Customer_Activity_Level	Tertile bucket (Low/Medium/High) of web visits + total purchases	Compresses raw activity volume into an ordinal engagement tier for easier segmentation and reporting.

Note on Total_Purchase and Total_Campaign: the exploratory notebook cells define Total_Purchase as the sum of all four purchase-count columns (including NumDealsPurchases) and Total_Campaign as the sum of four of the five campaign flags, while the reusable pipeline function (Task 7) defines Total_Purchase from three channel columns only (excluding deals) and Total_Campaign from all six campaign/response columns. Both versions are internally consistent but differ from each other; this is flagged in the Limitations section and should be reconciled to one definition before production use.

Task 2 — Categorical Feature Encoding

Four raw categorical columns plus one derived ordinal column required encoding before they could be used in a distance-based clustering algorithm.

Categorical columns identified	Education, Marital_Status, Preferred_Shopping_Channel, Product_Preference
Ordinal column identified	Customer_Activity_Level (Low / Medium / High)
Ordinal encoding	OrdinalEncoder — Education: Basic < 2N Cycle < Graduation < Master < PhD; Customer_Activity_Level: Low < Medium < High
Nominal encoding	LabelEncoder — Marital_Status, Preferred_Shopping_Channel, Product_Preference

Example — Education, original vs. encoded:

Graduation	2.0
PhD	4.0
Master	3.0
Basic	0.0
2N Cycle	1.0

Encoding caveat: Marital_Status, Preferred_Shopping_Channel, and Product_Preference are nominal (unordered) categories, but LabelEncoder assigns them arbitrary integer codes that a distance-based clustering algorithm will interpret as ordered and equally spaced. One-hot encoding would avoid imposing a false ordinal structure on these three columns and is recommended as a refinement before final clustering; it was not applied here to keep the feature count lower for a first-pass model.

Task 3 — Feature Selection

Seven columns were removed before clustering because they were identifiers, had been superseded by an engineered feature, or carried no variance:

ID	Unique identifier — no predictive or grouping value, and actively harmful to clustering distance calculations.
Year_Birth	Superseded by Customer_Age, which is more directly interpretable.
Kidhome / Teenhome	Superseded by Total_Children, which combines both into one signal.
Dt_Customer	Superseded by Customer_Tenure; raw date strings are not usable by clustering algorithms.
Z_CostContact / Z_Revenue	Constant columns (zero variance across all 2,240 rows) — no separating power for any clustering method.

After removing these seven columns and encoding the remainder, 39 features were retained for clustering, spanning demographics (Education, Marital_Status, Income, Age, Tenure, Family_Size), spending behavior (six Mnt* categories plus Total_Spending), purchase behavior (channel counts, Total_Purchase, Preferred_Shopping_Channel), engagement (Digital_Engagement, Customer_Activity_Level, NumWebVisitsMonth), and marketing response (five campaign-acceptance flags, Total_Campaign, Response, Complain).

Task 4 — Skewness and Feature Transformation

Skewness was computed for all 39 numeric features. Fifteen features exceeded the $|\text{skew}| > 0.5$ threshold and received a log1p transformation; the most extreme case, Average Spending per Purchase, started at a skewness of 4.88 and was reduced to -0.10 after transformation.

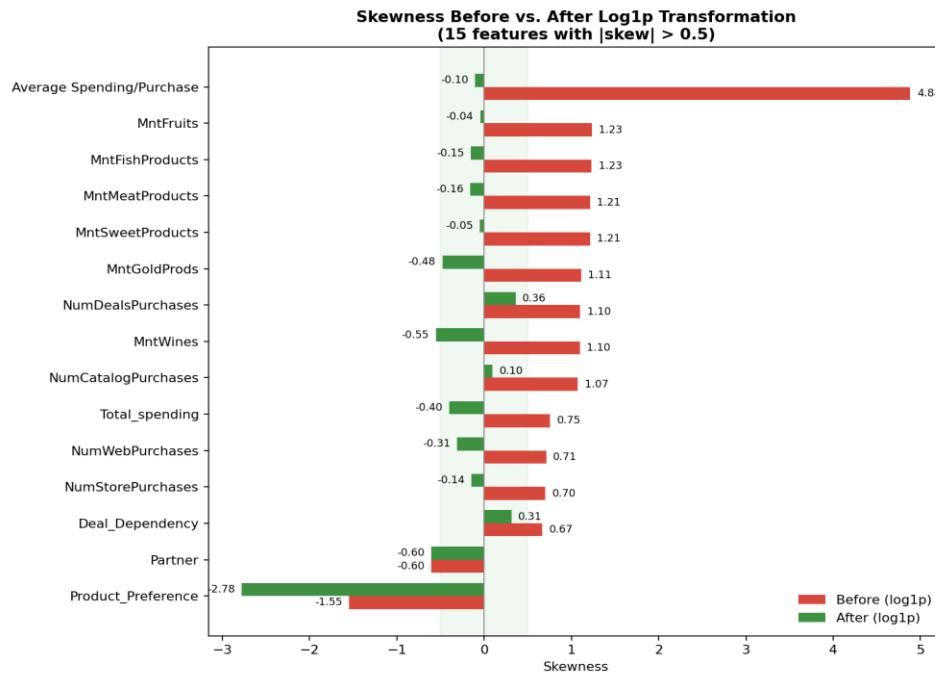


Figure 1. Skewness before vs. after log1p transformation for all 15 transformed features.

Two representative distributions before and after transformation:

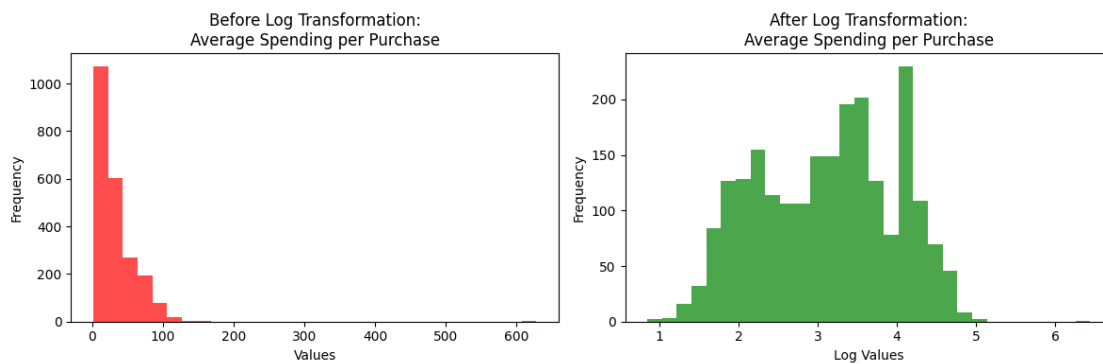


Figure 2. Average Spending per Purchase — before (red) and after (green) log1p transformation. The most dramatic correction in the dataset (skew $4.88 \rightarrow -0.10$).

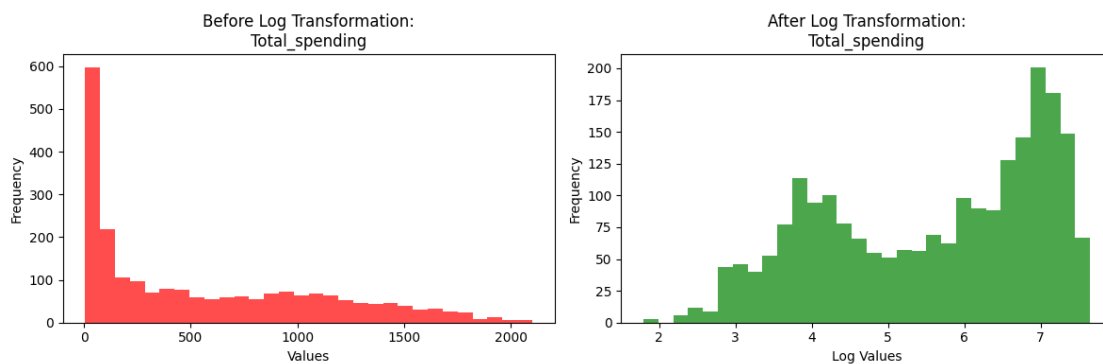


Figure 3. Total_Spending — before and after log1p transformation (skew $0.75 \rightarrow -0.40$).

Caveat — Product_Preference: this column is a label-encoded categorical variable (an arbitrary product-category ID, not a continuous quantity). Applying $\log_{1p}$ to it made its skewness worse ($-1.55 \rightarrow -2.78$) rather than better, because a monotonic numeric transform on category codes has no real quantitative meaning. Log transformation should be restricted to genuinely continuous, non-negative variables and skipped for encoded categorical IDs such as this one.

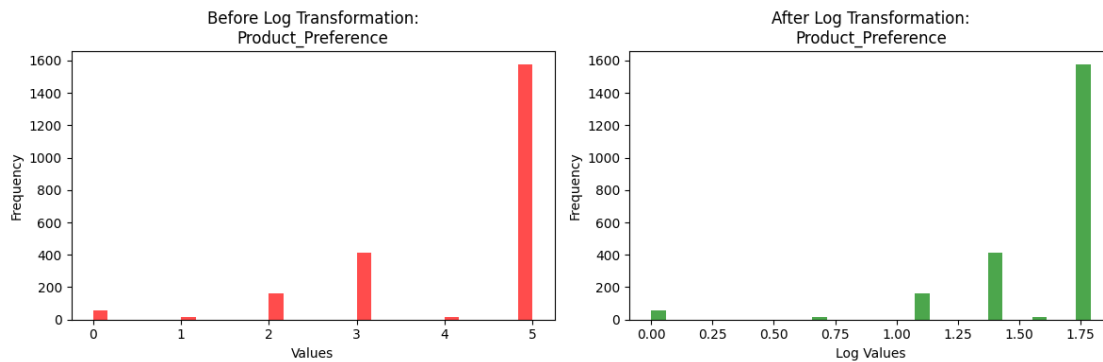


Figure 4. *Product_Preference* — $\log_{1p}$ transformation applied to a categorical code column; skewness worsened rather than improved.

Note on Partner: this binary-coded feature (values 1 or 2) shows identical skewness before and after transformation (-0.60 in both cases). This is expected: for a two-valued distribution, skewness depends only on the proportion of each value, not on the numeric labels themselves, so any monotonic re-encoding leaves it unchanged.

Task 5 — Feature Scaling

All 39 numeric features (post-transformation) were scaled with `StandardScaler`. The verification step confirmed every column now has mean ≈ 0 and standard deviation ≈ 1 (e.g., `Income`: mean 0.0037, std 1.0019; `MntWines`: mean -0.0014 , std 1.0014).

StandardScaler (chosen)	Centers to mean 0 / std 1. Well suited to K-Means and other Euclidean-distance clustering methods once skew has already been corrected by the Task 4 log transforms; assumes roughly symmetric distributions, which is now true for most features.
MinMaxScaler (not used)	Rescales to a fixed [0, 1] range. Very sensitive to remaining outliers (e.g., high-income or high-spending customers), which would compress the bulk of the data into a narrow band.
RobustScaler (considered)	Uses median / IQR instead of mean / std, more resistant to outliers. A reasonable alternative for <code>Income</code> and the <code>Mnt*</code> columns specifically, since a few high-value customers remain after log transformation; worth testing as a sensitivity check before finalizing the clustering model.

`StandardScaler` was selected as the primary approach because most of the skew-heavy outlier effects were already addressed in Task 4, making the remaining distributions close enough to symmetric for a mean/std-based scaler to work well for K-Means-style clustering.

Task 6 — Feature Engineering Documentation (Feature Dictionary)

The table below documents every engineered feature with its formula, purpose, business significance, and expected effect on the resulting customer segments.

Feature	Formula	Purpose	Business Significance	Expected Impact
Customer_Age	2014 – Year_Birth	Normalize age for segmentation	Age-based targeting (e.g., retirees vs. young families)	Helps clusters separate by life stage
Customer_Tenure	2014 – enrollment year	Measure loyalty duration	Identify long-term vs. new customers for retention offers	Separates loyal-customer clusters from recent sign-ups
Total_Children	Kidhome + Teenhome	Household composition summary	Family-oriented promotions and bundle sizing	Drives clusters around family-size-linked spending
Family_Size	Partner flag + Total_Children	Approximate household size	Sizing offers/packageing to household needs	Correlates with Total_Spending; helps separate large vs. small households
Total_Spending	Sum of 6 product-category Mnt* columns	Overall customer value	Core input for high-value vs. low-value segmentation	Primary driver of “value tier” clusters
Total_Purchase	Sum of purchase-count columns	Purchase frequency	Identify frequent vs. occasional buyers	Differentiates high-frequency from low-frequency clusters
Total_Campaign	Sum of accepted-campaign flags	Marketing responsiveness	Prioritize customers likely to respond to future campaigns	Separates campaign-responsive clusters
Average Spending per Purchase	Total_Spending / Total_Purchase	Basket-value profile	Distinguish bulk buyers from frequent small buyers	Adds a value-per-transaction dimension to clusters
Digital_Engagement	NumWebPurchases + NumWebVisitsMonth	Online engagement level	Guide digital marketing investment	Separates digitally engaged customers from offline-only ones
Deal_Dependency	NumDealsPurchases / Total_Purchase	Price sensitivity	Identify discount-driven customers for promo strategy	Flags a price-sensitive cluster distinct from full-price shoppers
Preferred_Shopping_Channel	arg-max of channel purchase counts	Primary shopping channel	Channel-specific campaign design	Enables channel-based cluster profiling
Product_Preference	arg-max of category spending	Primary product interest	Category-specific cross-sell/up-sell targeting	Adds product-affinity dimension to clusters
Customer_Activity_Level	Tertile of visits + purchases	Overall engagement tier	Quick low/medium/high engagement labeling for reporting	Provides an interpretable ordinal summary alongside numeric clusters

Task 7 — Preprocessing Pipeline Development

A reusable preprocessing_pipeline(df_raw, reference_year) function was developed to reproduce the feature-creation steps on any future raw extract with the same source columns:

```
def preprocessing_pipeline(df_raw, reference_year=2014):
    df = df_raw.copy()
    df['Income'] = df['Income'].fillna(df['Income'].median())
    df['Age'] = reference_year - df['Year_Birth']
    df['Customer_Tenure'] = reference_year - pd.to_datetime(df['Dt_Customer']).dt.year
    df['Total_Children'] =
```



```

= df['Kidhome'] + df['Teenhome']      df['Partner'] = df['Marital_Status'].apply(lambda x: 2 if x in
['Married','Together'] else 1)      df['Family_Size'] = df['Partner'] + df['Total_Children']
df['Total_spending'] = df[spending_cols].sum(axis=1)      df['Total_Purchase'] =
df[purchase_cols].sum(axis=1)      df['Total_Campaign'] = df[campaign_cols].sum(axis=1)      df['Average
Spending per Purchase'] = (df['Total_spending']/df['Total_Purchase']).fillna(0)      df['Digital_Engagement']
= df['NumWebPurchases'] + df['NumWebVisitsMonth']      df['Deal_Dependency'] =
(df['NumDealsPurchases']/df['Total_Purchase']).fillna(0)      df['Customer_Activity_Level'] =
pd.qcut(total_activity, q=3, labels=['Low','Medium','High'])      df['Preferred_Shopping_Channel'] =
df[purchase_cols].idxmax(axis=1)...      df['Product_Preference'] = df[spending_cols].idxmax(axis=1)...
return df

```

Coverage gap: as written, this function only covers missing-value handling and feature creation (steps 1–2 of the task specification). Encoding, log transformation, scaling, and final feature selection were performed as separate, manually-run notebook cells rather than being incorporated into the function itself. To be a fully reusable, single-call preprocessing pipeline as specified, the function — or a wrapping scikit-learn Pipeline/ColumnTransformer — should be extended to call the encoding, transformation, scaling, and selection steps internally, ideally with the encoders/scalers fit once and reused (via `.transform()`) on new data rather than re-fit each time.

Task 8 — Final Dataset Validation

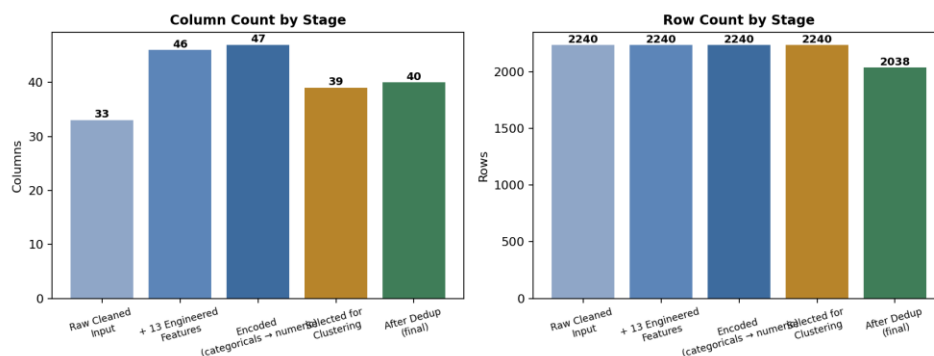


Figure 5. Column and row counts at each stage of the pipeline.

Missing values	0 across all 40 columns of the final dataset.
Duplicate records	202 duplicate rows were found once the ID column was dropped (0 duplicates were present with ID retained) — confirming these were genuinely identical customer profiles, not an artifact of the ID field. All 202 were removed.
Data types	All 40 columns are float64 after encoding and scaling; 0 object-type columns remain.
Row count change	2,240 → 2,038 rows (−9.0%) after de-duplication.
Scaling completion	Verified via <code>describe()</code> : every numeric column has mean ≈ 0 and std ≈ 1 .
Encoding completion	Verified via <code>select_dtypes(include='object')</code> returning zero columns.
Output file	Customer_Personality_Final_Dataset.csv

The dataset passes all eight validation checks and is machine-learning ready for the next stage (clustering / customer segmentation), subject to the encoding and pipeline-coverage refinements noted above.

Limitations & Recommendations

- **Inconsistent feature definitions:** Total_Purchase and Total_Campaign are computed differently in the exploratory cells versus the final pipeline function; reconcile to a single definition.
- **Nominal columns label-encoded:** Marital_Status, Preferred_Shopping_Channel, and Product_Preference should be one-hot encoded rather than label-encoded to avoid implying a false order/spacing between categories.
- **Log transform applied to a categorical code:** Product_Preference should be excluded from any further log-style transformation; only genuinely continuous columns should go through the $|\text{skew}| > 0.5$ rule.
- **Pipeline function is incomplete:** encoding, transformation, scaling, and selection are not yet inside the reusable function, limiting true one-call reusability on new data.
- **Duplicate rows were unexpected:** 202 rows (9% of the dataset) were exact duplicates once ID was removed. It is worth confirming with the data source whether these represent genuinely repeated customers, a data-collection artifact, or a merge/export issue before trusting cluster sizes downstream.